# Workflow and Tech Stack

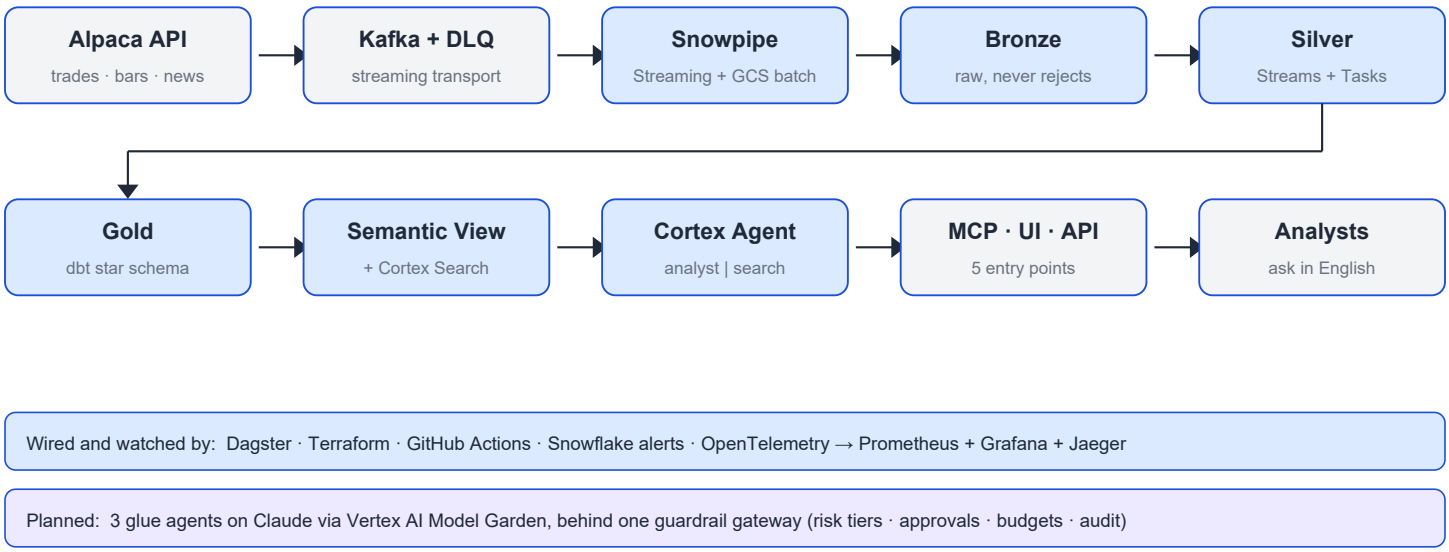
snowflake-ai-data-agent · real-time U.S. equities data on Snowflake, served to analysts through a governed AI agent, wired together with existing tools plus our own glue and guardrails

## What the project is

Live trades, minute bars and financial news flow from the Alpaca API through Kafka into Snowflake, where they are cleaned (Bronze → Silver), modelled into a star schema with dbt (Gold), and exposed through a semantic layer to Snowflake's Cortex Agent. Analysts ask questions in plain English through an MCP server, a Streamlit chat UI or a FastAPI service. Every path runs as the same role under the same row access policy, so the agent can never show a real-time price to someone not licensed to see it — everyone else sees prices delayed by 15 minutes.

The design principle is the one companies follow: **use existing tools for the heavy lifting** (Snowflake, dbt, Dagster, Terraform, GitHub Actions, Cortex Agent) and write only **glue** (what connects them) and **guardrails** (what keeps them and the AI safe). The goal is to remove repetitive human work while keeping human approval for anything that changes money, access or production.

**Status (14 Oct 2026):** verified locally — sample-data generators, DLQ routing, ONNX anomaly model, *dbt parse*, Dagster graph validation, *terraform validate*, and the local Kafka + observability stack (metrics, alerts and dashboard checked with replayed data); CI passes on GitHub. A Snowflake trial account exists and is being connected (key-pair auth set up). **No SQL has run against live Snowflake yet**; the glue agents and guardrail gateway are planned, not built.




*Grey = outside Snowflake · Blue = inside Snowflake / existing tools · Purple = our planned glue + guardrails*

# The workflow, end to end

Eleven stages in the order they happen, from creating the infrastructure to answering an analyst's question and responding when something breaks. Each row names the tool doing the work and the file that defines it.

## A. Provision the infrastructure (once)

| Step | What happens | Tool | Where in the repo |
|---|---|---|---|
| **A1** | Create the GCS bucket (no public access, 90-day file lifecycle), Pub/Sub topic + subscription, and the bucket notification that publishes every new file under raw/. | Terraform + Google provider | infra/terraform/gcp.tf |
| **A2** | Create the Snowflake database, the agent's warehouse ANALYST_WH (XSMALL, 30 s timeout), its credit monitor, six functional roles rolled up to SYSADMIN, and the GCS storage + Pub/Sub notification integrations. | Terraform + Snowflake provider | infra/terraform/snowflake.tf |
| **A3** | Grant Snowflake's own GCP service accounts read access to the bucket and subscription (replaces manual console steps). | Terraform | infra/terraform/gcp.tf |
| **A4** | Human step: review the Terraform plan and approve it. | GitHub Actions (production env) | .github/workflows/deploy.yml |

## B. Deploy the Snowflake objects (every release)

| Step | What happens | Tool | Where in the repo |
|---|---|---|---|
| **B1** | Bronze: RAW_TRADES, RAW_BARS, RAW_SYMBOLS — every column VARCHAR so a load never rejects a row. | Dagster → SQL | sql/02_bronze/bronze_tables.sql |
| **B2** | Silver tables + Streams + Tasks; news table + Cortex Search service. | Dagster → SQL | sql/03_silver, sql/04_documents |
| **B3** | Stage, file format and auto-ingest pipes (run after B1–B2: they load those tables). | Dagster → SQL | sql/01_ingest/snowpipe_setup.sql |
| **B4** | Roles, GOLD/STAGING schemas, DELAYED_DATA_POLICY, grants (incl. FUTURE grants); then guardrails: agent warehouse, credit monitor, append-only audit log. | Dagster → SQL | sql/07_governance, sql/09_guardrails |
| **B5** | dbt build: staging views, then DIM_SYMBOLS, DIM_DATE, FCT_TRADES, FCT_BARS; post-hooks attach the policy; tests must pass. | Dagster → dbt | dbt/ |
| **B6** | Semantic view → Cortex Agent → monitoring alerts. | Dagster → SQL | semantic_layer/, sql/10_agent, sql/11_monitoring |

The order is not the sql/ folder numbering: Dagster derives it from declared dependencies (orchestration/assets.py), so it can't drift. One command runs it all: *dagster job execute -m orchestration.definitions -j deploy_job*.

## C. Live ingestion (every second, market hours)

| Step | What happens | Tool | Where in the repo |
|---|---|---|---|
| **C1** | Websocket receives trades and minute bars; each message is validated against a Pydantic contract. | Python + Alpaca | streaming/alpaca_stream_producer.py, schemas.py |
| **C2** | Valid messages → Kafka topics market-trades / market-bars; malformed ones → market-data-dlq with the error. | Kafka | streaming/alpaca_stream_producer.py |
| **C3** | Kafka Connect streams rows into Bronze within seconds. | Snowpipe Streaming | streaming/snowflake_kafka_connector.json |
| **C4** | Offline alternative: replay the sample CSVs onto the same topics (no Alpaca account needed). | Python | streaming/replay_sample_data.py |
| **C5** | Both emit OpenTelemetry metrics: messages per topic, DLQ count, event-to-Kafka lag, time of last message. | OpenTelemetry | streaming/telemetry.py |

## D. Batch path (nightly, after the close)

| Step | What happens | Tool | Where in the repo |
|---|---|---|---|
| **D1** | Pull the day's bars, symbols and news from Alpaca REST into CSV. | Dagster → Python | streaming/backfill_historical.py, backfill_news.py |
| **D2** | Upload to gs://<bucket>/raw/<feed>/; the bucket publishes an event to Pub/Sub. | Dagster + GCS | orchestration/assets.py (gcs_raw_files) |
| **D3** | Snowpipe reads the event and loads the file into Bronze automatically; fills any gaps the stream left. | Snowpipe auto-ingest | sql/01_ingest/snowpipe_setup.sql |

## E. Transform

| Step | What happens | Tool | Where in the repo |
|---|---|---|---|
| **E1** | Every minute (trades, bars) / 15 min (symbols, news): a Task reads the Stream of new Bronze rows and MERGEs them into Silver — typed, deduplicated on business keys, symbols upper-cased. | Snowflake Tasks | sql/03_silver/streams_and_tasks.sql |
| **E2** | Every 5 minutes in market hours: dbt rebuilds Gold — returns, 30-bar rolling volatility, is_suspect flag for >10% one-minute moves; policy re-attached; tests run. | Dagster schedule → dbt | dbt/models/marts/ |
| **E3** | Blocking checks after each rebuild: policy still attached; failure stops anything downstream. | Dagster asset checks | orchestration/checks.py |

## F. Serve analysts

| Step | What happens | Tool | Where in the repo |
|---|---|---|---|
| **F1** | Semantic view defines metrics, dimensions and synonyms over Gold; verified queries anchor common questions. | Snowflake Semantic View | semantic_layer/ |
| **F2** | Cortex Agent routes each question: numbers → Cortex Analyst (SQL on the semantic view); "why did it move" → Cortex Search over news, with citations. | Snowflake Cortex Agent | sql/10_agent/ |
| **F3** | Five entry points, all as role ANALYST_AGENT: direct Analyst, Cortex Agent, MCP server (3 tools), Streamlit chat, FastAPI. | Python clients | agent/, ui/, api/ |

## G. Govern

| Step | What happens | Tool | Where in the repo |
|---|---|---|---|
| **G1** | DELAYED_DATA_POLICY: rows newer than 15 minutes visible only to REALTIME_DESK; compared in UTC with SYSDATE() so it doesn't depend on session timezone. | Row access policy | sql/07_governance/ |
| **G2** | Agent role is read-only on Gold; its warehouse has a 30 s timeout and a monthly credit cap; every question is written to an append-only audit log. | Roles, resource monitor | sql/09_guardrails/ |

## H. Monitor and respond

| Step | What happens | Tool | Where in the repo |
|---|---|---|---|
| **H1** | Snowflake ALERTs page on-call even if Dagster is down: ingestion stalled, policy detached (SEV-1); task failed, Gold stale (SEV-2). | Snowflake alerts + email | sql/11_monitoring/alerts.sql |
| **H2** | Guardrail checks every 15 minutes: policy attached, freshness, Bronze→Silver reconciliation, task health. | Dagster | orchestration/checks.py |
| **H3** | Outside Snowflake: the OTel Collector receives app metrics and traces; Prometheus scrapes it and kafka-exporter and alerts on producer stalled, DLQ > 1%, consumer lag > 10k, p95 event lag > 30 s. | OpenTelemetry, Prometheus | infra/local/ |
| **H4** | Grafana dashboard of the live path (localhost:3001); Jaeger for traces (localhost:16687). | Grafana, Jaeger | infra/local/grafana/ |
| **H5** | Human step: on-call follows the runbook; for governance incidents, pause the agent first. | Runbook | docs/runbook.md |

## I. Change a component

| Step | What happens | Tool | Where in the repo |
|---|---|---|---|
| **I1** | Branch, edit, push: CI compiles all Python, runs the generators, validates the problem catalog, parses dbt, loads the Dagster graph, and validates Terraform. | GitHub Actions | .github/workflows/ci.yml |
| **I2** | Human step: code review, then approve the production deploy (Terraform plan → apply → Dagster deploy_job). | GitHub Actions | .github/workflows/deploy.yml |

## J. Control cost and shut down

| Step | What happens | Tool | Where in the repo |
|---|---|---|---|
| **J1** | Account-wide cap: resource monitor stops every warehouse at 50 credits; warehouses auto-suspend after 60 s. | Resource monitor | sql/99_teardown/stop_spend.sql (level 0) |
| **J2** | Pause: suspend tasks, alerts and pipes; drop Cortex Search (costs credits while it exists). Remove: drop the database and integrations; terraform destroy. | SQL + Terraform | sql/99_teardown/stop_spend.sql (levels 1–2) |

## K. Planned: glue agents behind a guardrail gateway

| Step | What happens | Tool | Where in the repo |
|---|---|---|---|
| **K1** | Analyst glue: asks the Cortex Agent, checks the answer came from executed SQL, returns it with as-of time and citations, or escalates. | Claude on Vertex AI | docs/glue-and-guardrails.md |
| **K2** | Incident glue: on an alert, checks systems in order, names the first broken link with evidence, proposes the runbook step. | Claude on Vertex AI + MCP | docs/glue-and-guardrails.md |
| **K3** | Change glue: drafts the dbt model and tests, opens a PR with a stage report, deploys after approval. | Claude on Vertex AI + dbt/GitHub MCP | docs/modeling-agents-study.md |
| **K4** | Gateway on every tool call: read/draft automatic; reversible prod change = 1 approval; irreversible, access or money = approval + reviewed plan; secrets, weakening checks, widening access blocked; budgets; audit. | Python (planned) | docs/glue-and-guardrails.md §4 |
| **K5** | Model access: Claude through Vertex AI Model Garden in our GCP project (billing, IAM and audit in one cloud). Off by default; enabling the model in Model Garden is a human step. Analysts' questions stay on Snowflake Cortex so governed data never leaves Snowflake. | Vertex AI + Anthropic SDK | glue/llm.py, infra/terraform/vertex.tf |

# Freshness budget

| Hop | Target | Mechanism |
|---|---|---|
| **Alpaca → Kafka** | seconds | Websocket producer |
| **Kafka → Bronze** | seconds | Snowpipe Streaming |
| **Bronze → Silver** | ≤ 1 minute | Task every minute |
| **Silver → Gold** | ≤ 5 minutes | dbt every 5 minutes in market hours |
| **Gold → agent and non-entitled users** | +15 minutes (licensing, on purpose) | DELAYED_DATA_POLICY |
| **Visible to the agent** | ≈ 15–21 minutes; alert at 30 | — |
| **Visible to REALTIME_DESK** | ≈ 1–6 minutes | — |
| **Symbols / history / news** | daily / nightly / ≤ 1 hour | Batch on purpose: they don't need to be real-time |

# Technology stack

Versions are the ones installed and verified in this project's environment (Terraform providers pinned in infra/terraform/.terraform.lock.hcl). Snowflake and Alpaca are managed services, so they have no client version.

## Data platform (Snowflake features used)

| Feature | Used for | Where |
|---|---|---|
| **Snowpipe Streaming** | Row-level streaming from Kafka into Bronze in seconds | streaming/snowflake_kafka_connector.json |
| **Snowpipe (auto-ingest) + GCS storage & notification integrations** | Batch files from the bucket into Bronze | sql/01_ingest |
| **Streams + Tasks** | Change capture and scheduled MERGE, Bronze → Silver | sql/03_silver |
| **Dynamic Tables** | Alternative to Streams+Tasks, shown side by side (not wired in) | sql/03_silver/dynamic_tables_alternative.sql |
| **Semantic Views** | Governed metrics and dimensions the agent queries | semantic_layer/semantic_view.sql |
| **Cortex Analyst** | Natural language → SQL over the semantic model | agent/cortex_client.py |
| **Cortex Search** | Retrieval over news articles, with citations | sql/04_documents |
| **Cortex Agents** | Routes each question to Analyst or Search | sql/10_agent |
| **Row access policy** | 15-minute delayed view for non-entitled roles | sql/07_governance |
| **Roles + FUTURE grants + managed hierarchy** | Least privilege; grants survive dbt rebuilds | sql/07_governance, infra/terraform |
| **Resource monitors + warehouse timeouts** | Credit caps; agent warehouse isolation | sql/09_guardrails, sql/99_teardown |
| **ALERTs + email notification integration** | Paging that doesn't depend on the orchestrator | sql/11_monitoring |
| **Key-pair (JWT) authentication** | Tools connect without passwords | ~/.snowflake/connections.toml (local) |

## Tools and libraries

| Layer | Technology | Version | Used for | Where |
|---|---|---|---|---|
| **Language** | Python | 3.12.10 | Everything outside SQL | — |
| **Contracts** | Pydantic | 2.12.5 | Validating every record and agent response | streaming/schemas.py, agent/schemas.py |
| **Source** | Alpaca API (websocket + REST) | service | Trades, bars, news | streaming/ |
| **Streaming** | Apache Kafka (KRaft) + Kafka Connect | 4.1.0 (Docker) | Transport + DLQ | infra/local/, streaming/ |
| **Streaming client** | kafka-python · websockets | 3.0.11 · 16.1.1 | Producer and websocket | streaming/ |
| **Resilience** | tenacity · httpx | 9.1.4 · 0.28.1 | Retries with backoff; HTTP | streaming/, agent/ |
| **Transform** | dbt-core · dbt-snowflake | 1.12.5 · 1.12.1 | Gold star schema + tests | dbt/ |
| **Orchestration** | Dagster · dagster-dbt · dagster-snowflake | 1.13.25 · 0.29.25 · 0.29.25 | Deploy order, schedules, checks | orchestration/ |
| **Infrastructure** | Terraform · snowflakedb/snowflake · hashicorp/google | ≥1.6 · 2.21.0 · 7.46.1 | GCP + Snowflake account objects | infra/terraform/ |
| **Cloud storage** | Google Cloud Storage + Pub/Sub · google-cloud-storage | service · 3.16.0 | Batch landing zone + events | infra/terraform, orchestration/ |
| **Snowflake access** | snowflake-connector-python · Snowflake CLI | 4.7.5 · 3.28.0 | Queries; local admin | agent/, orchestration/ |
| **Agent serving** | fastmcp · streamlit · fastapi · uvicorn | 4.0.10 · 1.64.0 · 0.142.1 · 0.54.0 | MCP server, chat UI, HTTP API | agent/, ui/, api/ |
| **Tracing (LLM)** | Langfuse | 4.15.6 | Tracing Cortex agent calls | agent/ |

| Layer | Technology | Version | Used for | Where |
|---|---|---|---|---|
| **Instrumentation** | OpenTelemetry SDK · OTLP HTTP exporter · FastAPI instrumentation | 1.45.0 · 1.45.0 · 0.66b0 | Metrics + traces from producer, replay, API | streaming/telemetry.py, api/main.py |
| **Telemetry pipeline** | OpenTelemetry Collector (contrib) | 0.135.0 | Receives OTLP; exports to Prometheus + Jaeger | infra/local/otel-collector.yaml |
| **Metrics + alerts** | Prometheus · kafka-exporter | 3.6.0 · 1.9.0 | Scraping; 4 alert rules | infra/local/ |
| **Dashboards** | Grafana | 12.2.0 | Live-pipeline dashboard | infra/local/grafana/ |
| **Traces** | Jaeger | 2.10.0 | Trace UI | infra/local/ |
| **Local runtime** | Docker Compose | — | Kafka + observability on one command | infra/local/docker-compose.yml |
| **ML** | scikit-learn · skl2onnx · onnx | 1.9.1 · 1.20.0 · 1.23.1 | IsolationForest anomaly model → ONNX | snowpark/train_anomaly_model.py |
| **Data handling** | pandas | 3.0.6 | Profiling, model training | snowpark/ |
| **CI/CD** | GitHub Actions | service | CI every push; approval-gated deploy | .github/workflows/ |
| **Docs** | reportlab · python-docx · pypdfium2 · PyYAML | 5.0.1 · 1.2.0 · 5.13.0 · 6.0.2 | These PDF and Word documents; the catalog | docs/, knowledge/ |
| **Models (glue agents)** | Claude on Vertex AI Model Garden · Anthropic SDK [vertex] | service · 1.13.0 | Glue agents behind the guardrail gateway (off by default) | glue/llm.py, infra/terraform/vertex.tf |

## Who can see what

| Role | Purpose | Sees real-time prices? |
|---|---|---|
| **LOADER** | Ingestion (streaming + batch) | n/a (writes Bronze only) |
| **TRANSFORMER** | dbt; can attach the policy (APPLY) | n/a (builds Gold) |
| **ANALYST_AGENT** | The AI agent and all five entry points | No — 15 minutes delayed |
| **BI_READER** | Human dashboards | No — 15 minutes delayed |
| **REALTIME_DESK** | Entitled users (e.g. a trading desk) | Yes |
| **AUDIT_REVIEWER** | Reads the agent audit log only | n/a |

## Where humans stay in the loop

Repetitive work is automated; these decisions stay human on purpose:

- Approve the Terraform plan and every production deploy.
- Review code changes (CI must be green first).
- Grant real-time entitlement (REALTIME_DESK).
- Approve new metric definitions in the semantic layer.
- Respond to SEV-1 pages — pausing the agent is the safe default.
- Planned gateway: approve reversible production actions; approve + review irreversible ones.

# What's verified, written, and planned

| Component | Status |
|---|---|
| **Sample data generators, DLQ routing, ONNX model training** | Verified locally (run against sample data) |
| **CI pipeline** | Passing on GitHub |
| **dbt project** | dbt parse passes; not run against Snowflake |
| **Dagster asset graph and schedules** | dagster definitions validate passes; not run against Snowflake |
| **Terraform** | terraform validate passes against real providers; not applied |
| **All SQL (Bronze → Agent, governance, alerts, stop-spend)** | Written; not yet run on the live trial |
| **Snowflake trial account + key-pair connection** | Account active; key registration in progress |
| **Local Kafka + observability stack (OTel, Prometheus, Grafana, Jaeger)** | Verified locally: replayed data reached Prometheus and matched Kafka offsets; alerts and dashboard load |
| **Vertex AI wiring** | Client builds and terraform validates; no model call made (needs Model Garden enablement + budget) |
| **Approval-gated deploy workflow** | Written; not run |
| **Glue agents + guardrail gateway** | Planned (docs/glue-and-guardrails.md) |
| **pytest suite** | Not built yet |

# How to run it

| Goal | Command |
|---|---|
| **Install** | pip install -r requirements.txt |
| **Start Kafka + observability** | cd infra/local && docker compose up -d |
| **Send telemetry** | set OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:14318, then run the producer, replay or API |
| **Watch it** | Grafana localhost:3001 · Prometheus localhost:9091 · Jaeger localhost:16687 |
| **Generate sample data** | python streaming/generate_sample_data.py · python streaming/generate_sample_news.py |
| **Provision infrastructure** | cd infra/terraform && terraform apply |
| **Deploy everything in order** | dagster job execute -m orchestration.definitions -j deploy_job |
| **Run schedules + UI** | dagster dev -m orchestration.definitions (localhost:3000) |
| **Build Gold by hand** | cd dbt && dbt build |
| **Ask the agent** | python agent/cortex_agent_client.py "What's happening with AAPL today?" |
| **Chat UI / API** | cd ui && streamlit run streamlit_app.py · cd api && uvicorn main:app |
| **Check Vertex AI wiring (no cost)** | python -m glue.llm --check |
| **Cap / pause spend** | Run sql/99_teardown/stop_spend.sql (levels 0–1) in Snowsight |
| **Rebuild these PDFs** | python docs/build_workflow_doc.py · python docs/build_playbook.py |

## Further reading in the repo

- README.md — full technical reference, repository map and feature status.
- docs/production-readiness.md — existing tools vs our glue and guardrails; human work removed.
- docs/glue-and-guardrails.md — plan of record for the glue agents and gateway.
- docs/runbook.md — on-call procedures, one entry per alert.
- docs/snowflake-e2e-agent-playbook.pdf + knowledge/problem_catalog.yaml — 158 real-world problems and who handles each.